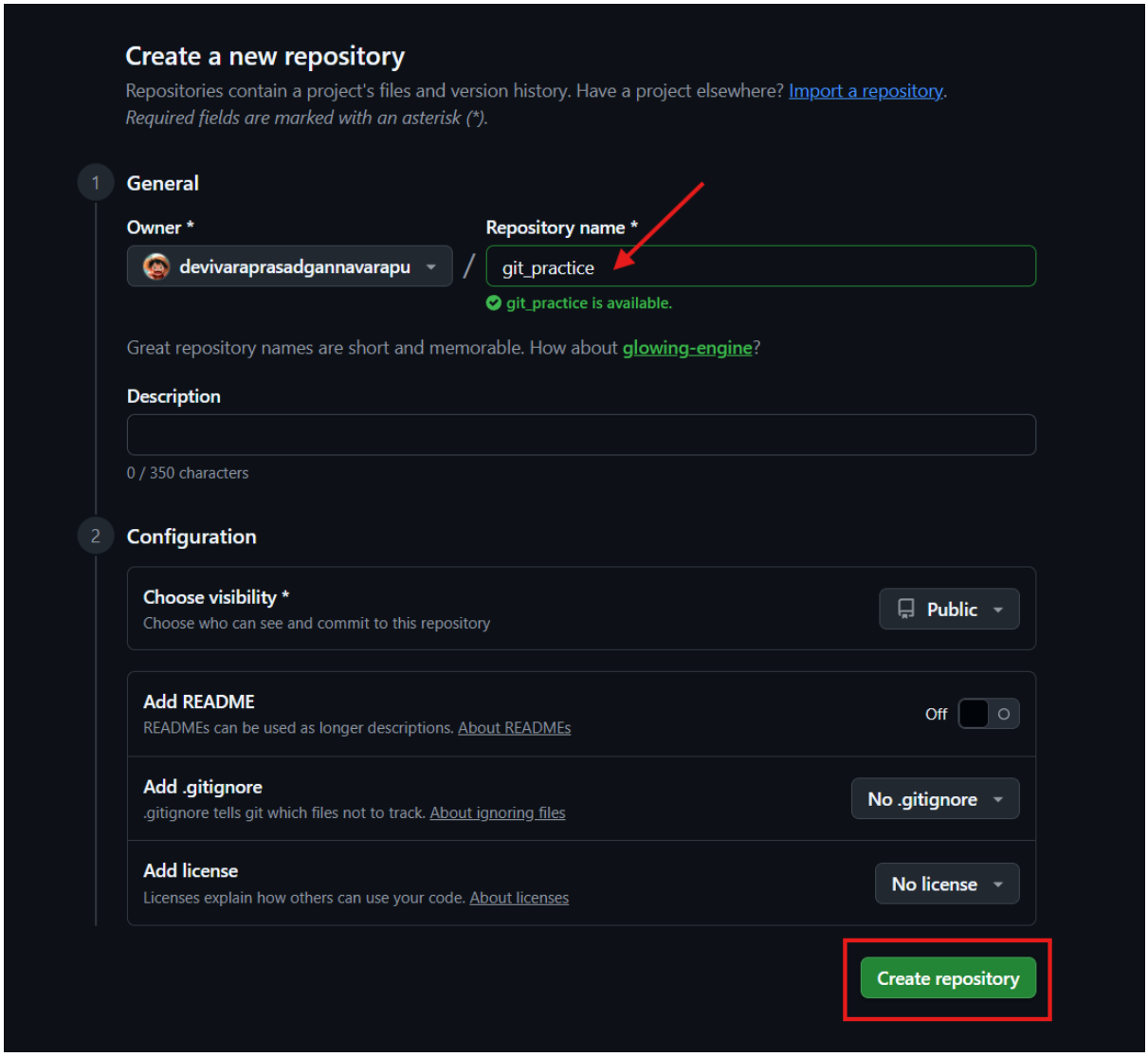
Task 1: Branching & Merging

- Initialize a new Git repo called **git_practice**.
- Create a new branch feature/login.
- Add a file login.sh and commit changes.
- Switch to main, create another file readme.md, and commit.
- Merge feature/login into main.
- Resolve any merge conflicts if they occur.
- Push all branches to GitHub.

Step 1. Create repository on Git Hub

I created a repository named **git_practice** on GitHub.



Step 2. Initialize a new Git repo called git_practice.

Executing these commands in git_practice directory

git init

```
[ec2-user@ip-172-31-15-12 ~]$ mkdir git_practice
[ec2-user@ip-172-31-15-12 ~]$ cd git_practice/
[ec2-user@ip-172-31-15-12 git_practice]$ #Initialize the dirictory
[ec2-user@ip-172-31-15-12 git_practice]$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/ec2-user/git_practice/.git/
[ec2-user@ip-172-31-15-12 git_practice]$
```

echo "git_practice read file" > readme.txt

git add readme.txt

git commit -m "readme.txt file commited"

```
[ec2-user@ip-172-31-15-12 git_practice]$ # without any file in the main branch, feature branch not created
[ec2-user@ip-172-31-15-12 git_practice]$ echo "git_practice read file" > readme.txt
[ec2-user@ip-172-31-15-12 git_practice]$ git add readme.txt
[ec2-user@ip-172-31-15-12 git_practice]$ git commit -m "readme.txt file commited"
[master (root-commit) 6fa39c5] readme.txt file commited
 1 file changed, 1 insertion(+)
 create mode 100644 readme.txt
[ec2-user@ip-172-31-15-12 git_practice]$
```

Step 3. Rename default branch to main and add remote

git branch -M main

git remote add origin https://github.com/devivaraprasadgannavarapu/git_practice.git

git push -u origin main

```
[ec2-user@ip-172-31-15-12 git_practice]$ git branch -M main
[ec2-user@ip-172-31-15-12 git_practice]$ git remote add origin https://github.com/devivaraprasadgannavarapu/git_practice.git
[ec2-user@ip-172-31-15-12 git_practice]$ git push -u origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 250 bytes | 250.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/devivaraprasadgannavarapu/git_practice.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
[ec2-user@ip-172-31-15-12 git_practice]$
```

Step 4. Create branch feature/login, and add a file [login.sh] and commit changes.

git checkout -b feature_login

echo "login page" > login.sh

git add login.sh

git commit -m "login.sh Added"

```
[ec2-user@ip-172-31-15-12 git_practice]$ git checkout -b feature_login
Switched to a new branch 'feature_login'
[ec2-user@ip-172-31-15-12 git_practice]$ echo "login page" > login.sh
[ec2-user@ip-172-31-15-12 git_practice]$ git add login.sh
[ec2-user@ip-172-31-15-12 git_practice]$ git commit -m "login.sh Added"
[feature_login 6ce7d2d] login.sh Added
 1 file changed, 1 insertion(+)
 create mode 100644 login.sh
[ec2-user@ip-172-31-15-12 git_practice]$
```

Step 5. Switch to main, create another file [readme.txt] and commit.

git switch main

echo "added some more lines" >> readme.txt

git add readme.txt

git commit -m "updated the readme.txtfile"

```
[ec2-user@ip-172-31-15-12 git_practice]$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
[ec2-user@ip-172-31-15-12 git_practice]$ echo "added some more lines" >> readme.txt
[ec2-user@ip-172-31-15-12 git_practice]$ git add readme.txt
[ec2-user@ip-172-31-15-12 git_practice]$ git commit -m "updated the readme.txtfile"
[main 77cfcc8] updated the readme.txtfile
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-15-12 git_practice]$
```

Step 6. Merge feature-login into main

```
[ec2-user@ip-172-31-15-12 git_practice]$ git merge feature_login
Merge made by the 'ort' strategy.
 login.sh | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 login.sh
[ec2-user@ip-172-31-15-12 git_practice]$
```

- If merge conflicts occur, resolve in the affected files, git add the resolved files, then git commit.

Step 7. Push branches to GitHub

git push origin main

```
[ec2-user@ip-172-31-15-12 git_practice]$ git push origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (8/8), 787 bytes | 787.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/devivaraprasadgannavarapu/git_practice.git
   6fa39c5..53d8aac  main -> main
[ec2-user@ip-172-31-15-12 git_practice]$
```

git push origin feature_login

```
[ec2-user@ip-172-31-15-12 git_practice]$ git push origin feature_login
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature_login' on GitHub by visiting:
remote:      https://github.com/devivaraprasadgannavarapu/git_practice/pull/new/feature_login
remote:
To https://github.com/devivaraprasadgannavarapu/git_practice.git
 * [new branch]      feature_login -> feature_login
[ec2-user@ip-172-31-15-12 git_practice]$
```

Task 2: Reverse & Reset

- Create a Git repo git_reset_revert_demo.
- Add 3 commits by modifying a file demo.txt.
- Use git log to identify commit hashes.
- Use git revert to undo the second commit without removing history.
- Then use git reset --hard to go back to the first commit.
- Push final repo to GitHub and share screenshot of git log --oneline.

Step 1. Clone the Repository

git clone https://github.com/devivaraprasadgannavarapu/git_reset_revert_demo.git

cd git_reset_revert_demo

git log --oneline

```
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git log --oneline
e5000f6 (HEAD -> main, origin/main, origin/HEAD) Initial commit
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$
```

Step 2. Create demo.txt and Make 3 Commits

```
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ echo "demo file first line" >> demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git add demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git commit -m "first commit d1"
[main 358de39] first commit d1
 1 file changed, 1 insertion(+)
 create mode 100644 demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ echo "demo file second line" >> demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git add demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git commit -m "second commit d2"
[main 9d6003d] second commit d2
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ echo "demo file third line" >> demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git add demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git commit -m "third commit d3"
[main 13ffd8d] third commit d3
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$
```

Step 3. Commit hashes

```
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git log --oneline
13ffd8d (HEAD -> main) third commit d3
9d6003d second commit d2
358de39 first commit d1
e5000f6 (origin/main, origin/HEAD) Initial commit
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$
```

Step 4. Revert the second commit

```
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git revert 9d6003d
Auto-merging demo.txt
CONFLICT (content): Merge conflict in demo.txt
error: could not revert 9d6003d... second commit d2
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git revert --continue".
hint: You can instead skip this commit with "git revert --skip".
hint: To abort and get back to the state before "git revert",
hint: run "git revert --abort".
hint: Disable this message with "git config advice.mergeConflict false"
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ cat demo.txt
demo file first line
<<<<<<< HEAD
demo file second line
demo file third line
=======
>>>>>>> parent of 9d6003d (second commit d2)
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ vi demo.txt
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git add .
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git revert --continue
[main 876f5d2] Revert "second commit d2"
 1 file changed, 1 deletion(-)
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git log --oneline
876f5d2 (HEAD -> main) Revert "second commit d2"
13ffd8d third commit d3
9d6003d second commit d2
358de39 first commit d1
e5000f6 (origin/main, origin/HEAD) Initial commit
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$
```

Step 5. Reset Hard to first commit (d1)

```
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git reset --hard 358de39
HEAD is now at 358de39 first commit d1
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git log --oneline
358de39 (HEAD -> main) first commit d1
e5000f6 (origin/main, origin/HEAD) Initial commit
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$
```

Step 6. Push final Repo to GitHub

git push origin main

```
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git push origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 303 bytes | 303.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/devivaraprasadgannavarapu/git_reset_revert_demo.git
   e5000f6..358de39  main -> main
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$
```

Step 7. Final Git log

git log –oneline

```
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$ git log --oneline
358de39 (HEAD -> main, origin/main, origin/HEAD) first commit d1
e5000f6 Initial commit
[ec2-user@ip-172-31-15-12 git_reset_revert_demo]$
```

Task 3: Git Stash Scenario

- Create a repo stash_practice.
- Modify two files and don't commit them.
- Run git stash to temporarily save your work.
- Switch to a new branch bugfix and make a new commit.
- Return to main and apply your stash using git stash apply.
- Push final code and share output of git stash list.

Step 1. Create Repository

mkdir stash_practice

cd stash-practice

git init

echo "Stash practice" >> readme.md

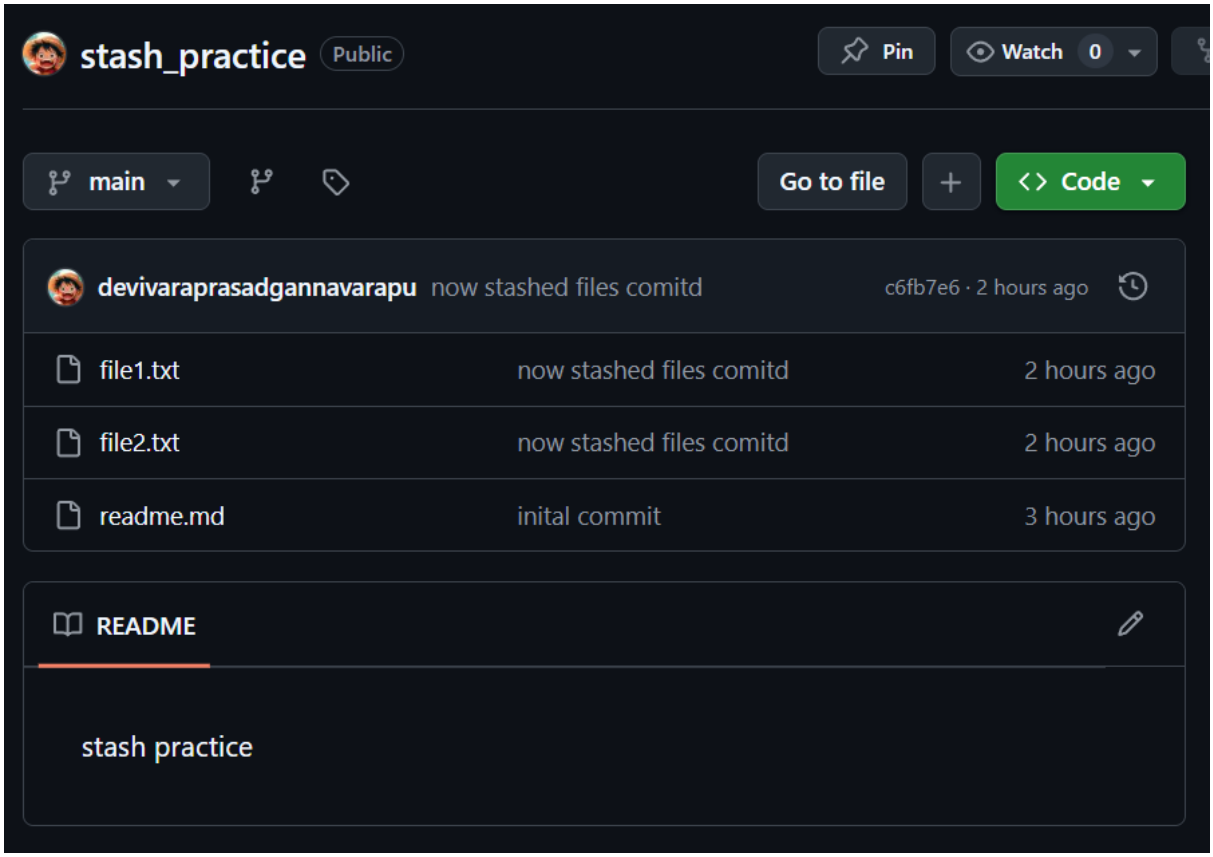
git add .

git commit -m "Initial commit"

git branch -M main

git remote add origin https://github.com/devivaraprasadgannavarapu/stash_practice.git

git push origin main



Step 2. Create and commit 2 files

echo " this is file 1" >> file1.txt

echo " this is file 2" >> file2.txt

git add .

git commit -m " two files commited"

git push origin main

```
[ec2-user@ip-172-31-15-12 stash_practice]$ echo "this is file 1" >> file1.txt
[ec2-user@ip-172-31-15-12 stash_practice]$ echo "this is file 2" >> file2.txt
[ec2-user@ip-172-31-15-12 stash_practice]$ git add .
[ec2-user@ip-172-31-15-12 stash_practice]$ git log --oneline
6abb56e (HEAD -> main, origin/main) inital commit
[ec2-user@ip-172-31-15-12 stash_practice]$
```

```
[ec2-user@ip-172-31-15-12 stash_practice]$ git commit -m "two files commited"
[main 22d21fb] two files commited
 2 files changed, 2 insertions(+)
 create mode 100644 file1.txt
 create mode 100644 file2.txt
[ec2-user@ip-172-31-15-12 stash_practice]$ git push origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 350 bytes | 350.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/devivaraprasadgannavarapu/stash_practice.git
   6abb56e..22d21fb  main -> main
```

Step 3. Modify and Stash the files

echo “modifying file 1” >> file1.txt

echo “modifying file 2” >> file2.txt

```
[ec2-user@ip-172-31-15-12 stash_practice]$ echo “modifying file 1” >> file1.txt
[ec2-user@ip-172-31-15-12 stash_practice]$ echo “modifying file 2” >> file2.txt
[ec2-user@ip-172-31-15-12 stash_practice]$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt
        modified:   file2.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-15-12 stash_practice]$
```

git stash

git stash list

```
[ec2-user@ip-172-31-15-12 stash_practice]$ git stash
Saved working directory and index state WIP on main: 22d21fb two files commited
[ec2-user@ip-172-31-15-12 stash_practice]$ git stash list
stash@{0}: WIP on main: 22d21fb two files commited
```

Step 4. Create and Switch to a new branch then make a new commit

git switch -c bug_fix

echo “bug fixing” >> bugfix.txt

git add .

git commit -m “bug was fixed”

```
[ec2-user@ip-172-31-15-12 stash_practice]$ git switch -c bug_fix
Switched to a new branch 'bug_fix'
[ec2-user@ip-172-31-15-12 stash_practice]$ echo "bug fixing" >> bugfix.txt
[ec2-user@ip-172-31-15-12 stash_practice]$ ls
bugfix.txt  file1.txt  file2.txt  readme.md
[ec2-user@ip-172-31-15-12 stash_practice]$ git add .
[ec2-user@ip-172-31-15-12 stash_practice]$ git commit -m "bug was fixed"
[bug_fix 345369d] bug was fixed
 1 file changed, 1 insertion(+)
 create mode 100644 bugfix.txt
[ec2-user@ip-172-31-15-12 stash_practice]$
```

Step 5. Apply Stash

git switch main

git stash apply

git add .

git commit -m "now stashed files comitd"

git push origin main

```
[ec2-user@ip-172-31-15-12 stash_practice]$ git  switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
[ec2-user@ip-172-31-15-12 stash_practice]$ git stash apply
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt
        modified:   file2.txt

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-15-12 stash_practice]$ git add .
[ec2-user@ip-172-31-15-12 stash_practice]$ git commit -m "now stashed files comitd"
[main c6fb7e6] now stashed files comitd
 2 files changed, 2 insertions(+)
[ec2-user@ip-172-31-15-12 stash_practice]$ git push origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 392 bytes | 392.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/devivaraprasadgannavarapu/stash_practice.git
   22d21fb..c6fb7e6  main -> main
[ec2-user@ip-172-31-15-12 stash_practice]$
```

Task 4: Cherry-pick Practice

- Create repo cherrypick_demo.
- Create a branch featureA and make 3 commits.
- Switch to main and cherry-pick only the 2nd commit from featureA.
- Verify using git log that only that commit was added.
- Push to GitHub and share screenshots of logs before & after cherry-pick.

Step 1. Create a new Git repository

mkdir cherrypick_demo

cd cherrypick_demo

echo "# cherrypick_demo" >> README.md

git init

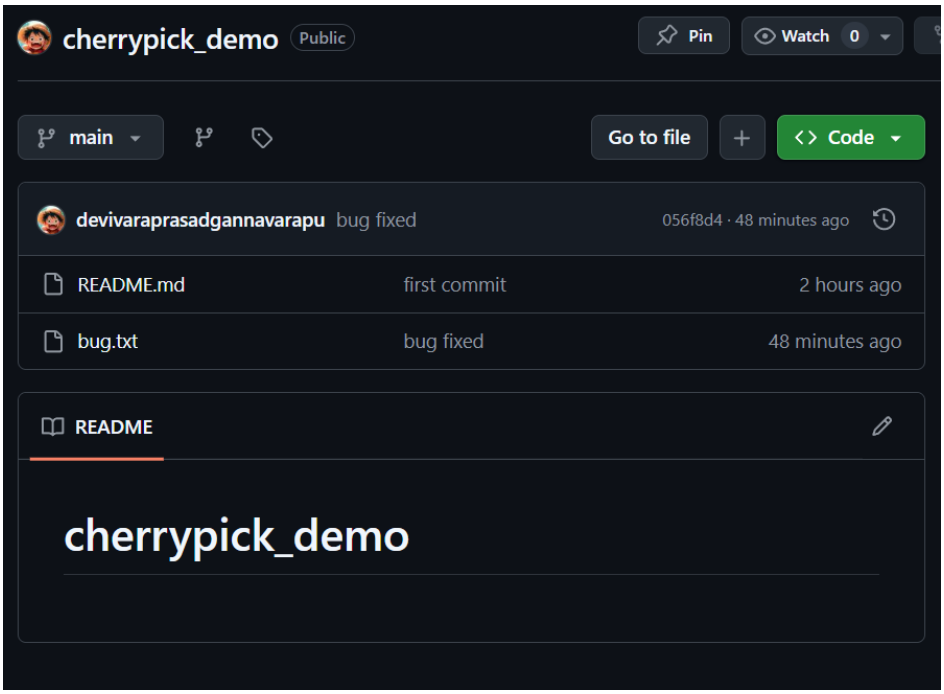
git add README.md

git commit -m "first commit"

git branch -M main

git remote add origin https://github.com/devivaraprasadgannavarapu/cherrypick_demo.git

git push -u origin main



Step 2. Create a new branch feature-A and add 3 commits

```
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git log --oneline
c986ee7 (HEAD -> main, origin/main, origin/HEAD) first commit
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git checkout -b feature_A
Switched to a new branch 'feature_A'
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ echo "first file " >> file1.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ echo "first file  line" >> file1.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git add .
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git commit -m "first commit in branch A"
[feature_A 8f5c2c2] first commit in branch A
 1 file changed, 2 insertions(+)
 create mode 100644 file1.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ echo "fixing bug" >> bug.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git add .
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git commit -m "bug fixed"
[feature_A 1d0f52d] bug fixed
 1 file changed, 1 insertion(+)
 create mode 100644 bug.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ echo "second file" >> file2.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git add .
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git commit -m "second commt file"
[feature_A fbbfd3c] second commt file
 1 file changed, 1 insertion(+)
 create mode 100644 file2.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$
```

Step 3. Switching to main branch

git switch main

```
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git log --oneline
fbbfd3c (HEAD -> feature_A) second commt file
1d0f52d bug fixed
8f5c2c2 first commit in branch A
c986ee7 (origin/main, origin/HEAD, main) first commit
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

Step 4. Cherry-pick only the second commit (1d0f52d) from feature-A

git cherry-pick 1d0f52d

```
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git cherry-pick 1d0f52d
[main 056f8d4] bug fixed
 Date: Fri Mar 6 11:32:24 2026 +0000
 1 file changed, 1 insertion(+)
 create mode 100644 bug.txt
[ec2-user@ip-172-31-15-12 cherrypick_demo]$
```

Step 5. Check commit added to main branch

git log –oneline

```
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git log --oneline
056f8d4 (HEAD -> main) bug fixed
c986ee7 (origin/main, origin/HEAD) first commit
[ec2-user@ip-172-31-15-12 cherrypick_demo]$
```

Step 6. Push changes to the GitHub

git push origin main

```
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ git push origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 294 bytes | 294.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/devivaraprasadgannavarapu/cherrypick_demo.git
   c986ee7..056f8d4  main -> main
[ec2-user@ip-172-31-15-12 cherrypick_demo]$ |
```

Task 5: Rebase scenario

- Create a repo rebase_practice.
- Create branches main and feature/db.
- Add a few commits in both branches.
- Rebase feature/db on top of main.
- Resolve any conflicts and complete the rebase.
- Push both branches and share git log --graph --oneline --all.

Step 1. Create a new Git repository

mkdir rebase_practice

cd rebase_practice

echo "# rebase_practice" >> README.md

git init

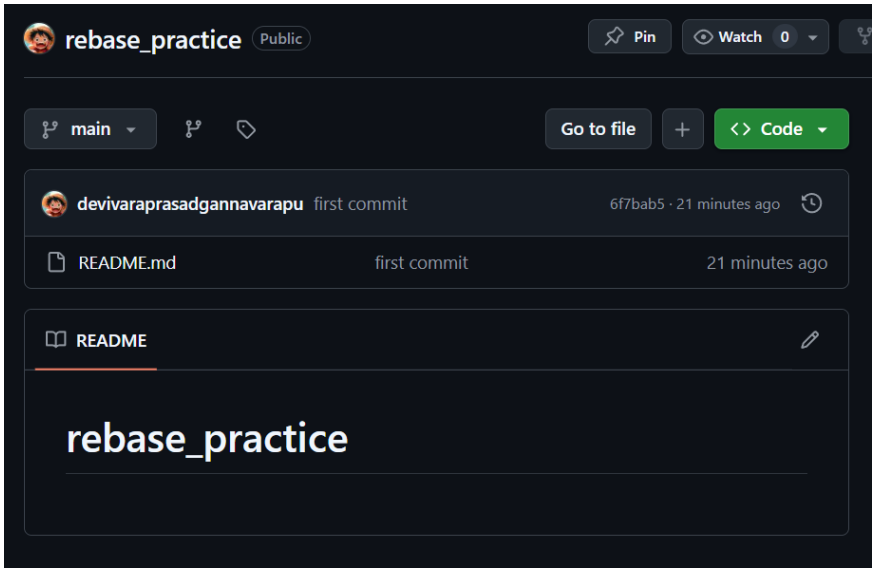
git add README.md

git commit -m "first commit"

git branch -M main

git remote add origin https://github.com/devivaraprasadgannavarapu/rebase_practice.git

git push -u origin main



Step 2. Create feature branch (feature_db)

git branch feature_db

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ git branch feature_db
[ec2-user@ip-172-31-15-12 rebase_practice]$ git branch
  feature_db
* main
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

Step 3. Add 3 commits to main branchs

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ git branch feature_db
[ec2-user@ip-172-31-15-12 rebase_practice]$ git branch
  feature_db
* main
[ec2-user@ip-172-31-15-12 rebase_practice]$ echo "1st commit in  main" >> c1
[ec2-user@ip-172-31-15-12 rebase_practice]$ git add .
[ec2-user@ip-172-31-15-12 rebase_practice]$ git commit -m "main c1"
[main 6bc9bce] main c1
 1 file changed, 1 insertion(+)
 create mode 100644 c1
[ec2-user@ip-172-31-15-12 rebase_practice]$ echo "2nd commit in  main" >> c2
[ec2-user@ip-172-31-15-12 rebase_practice]$ git add .
[ec2-user@ip-172-31-15-12 rebase_practice]$ git commit -m "main c2"
[main 5106b6e] main c2
 1 file changed, 1 insertion(+)
 create mode 100644 c2
[ec2-user@ip-172-31-15-12 rebase_practice]$ echo "3rd commit in  main" >> c3
[ec2-user@ip-172-31-15-12 rebase_practice]$ git add .
[ec2-user@ip-172-31-15-12 rebase_practice]$ git commit -m "main c3"
[main 71c66b4] main c3
 1 file changed, 1 insertion(+)
 create mode 100644 c3
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

git log --oneline

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ git log --oneline
71c66b4 (HEAD -> main) main c3
5106b6e main c2
6bc9bce main c1
6f7bab5 (origin/main, feature_db) first commit
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

Step 4. Switch to feature_db branch and add 3 commits

git switch feature_db

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ git switch feature_db
Switched to branch 'feature_db'
[ec2-user@ip-172-31-15-12 rebase_practice]$ echo "4th commit in feature_db" >> c4
[ec2-user@ip-172-31-15-12 rebase_practice]$ git add .
[ec2-user@ip-172-31-15-12 rebase_practice]$ git commit -m "feature_db c4"
[feature_db fa0ae20] feature_db c4
 1 file changed, 1 insertion(+)
 create mode 100644 c4
[ec2-user@ip-172-31-15-12 rebase_practice]$ echo "5th commit in feature_db" >> c5
[ec2-user@ip-172-31-15-12 rebase_practice]$ git add .
[ec2-user@ip-172-31-15-12 rebase_practice]$ git commit -m "feature_db c5"
[feature_db 3544ff2] feature_db c5
 1 file changed, 1 insertion(+)
 create mode 100644 c5
[ec2-user@ip-172-31-15-12 rebase_practice]$ echo "6th commit in feature_db" >> c6
[ec2-user@ip-172-31-15-12 rebase_practice]$ git add .
[ec2-user@ip-172-31-15-12 rebase_practice]$ git commit -m "feature_db c6"
[feature_db 976018d] feature_db c6
 1 file changed, 1 insertion(+)
 create mode 100644 c6
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

git log --oneline

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ git log --oneline
976018d (HEAD -> feature_db) feature_db c6
3544ff2 feature_db c5
fa0ae20 feature_db c4
6f7bab5 (origin/main) first commit
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

Step 5. Rebase feature_db on top of main

git rebase main

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ #rebase feature_db on top of main
[ec2-user@ip-172-31-15-12 rebase_practice]$ git branch
* feature_db
  main
[ec2-user@ip-172-31-15-12 rebase_practice]$ git rebase main
Successfully rebased and updated refs/heads/feature_db.
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

Step 6. Verify commit history

git log –oneline –graph

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ git log --oneline --graph
* 5eee48f (HEAD -> feature_db) feature_db c6
* 8cc27c7 feature_db c5
* 58bd6ea feature_db c4
* 71c66b4 (main) main c3
* 5106b6e main c2
* 6bc9bce main c1
* 6f7bab5 (origin/main) first commit
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

Step 7. Push both branches and see log graph

git push origin main

git push origin feature_db

```
[ec2-user@ip-172-31-15-12 rebase_practice]$ git push origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 747 bytes | 747.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/devivaraprasadgannavarapu/rebase_practice.git
   6f7bab5..71c66b4  main -> main
[ec2-user@ip-172-31-15-12 rebase_practice]$ git push origin feature_db
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 864 bytes | 864.00 KiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
remote:
remote: Create a pull request for 'feature_db' on GitHub by visiting:
remote:      https://github.com/devivaraprasadgannavarapu/rebase_practice/pull/new/feature_db
remote:
To https://github.com/devivaraprasadgannavarapu/rebase_practice.git
 * [new branch]      feature_db -> feature_db
[ec2-user@ip-172-31-15-12 rebase_practice]$
```

Task 6: Tagging & Release

- Create a repo release_versioning.
- Make 3 commits (v1 feature, v2 feature, v3 feature).
- Tag each commit as v1.0, v2.0, v3.0.
- Push all tags to GitHub.
- Share output of git tag -l and screenshot from GitHub showing **releases.**

Step 1. Create a new Git repository.

mkdir release_versioning

cd release_versioning

echo "# release_versioning" >> README.md

git init

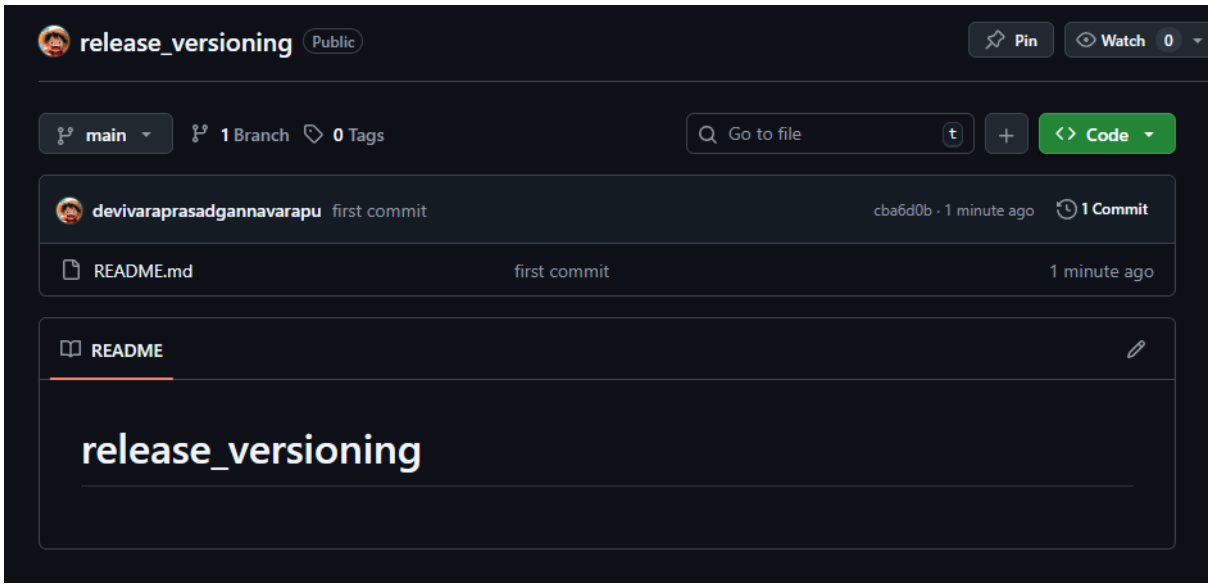
git add README.md

git commit -m "first commit"

git branch -M main

git remote add origin https://github.com/devivaraprasadgannavarapu/release_versioning.git

git push -u origin main



Step 2. Add 3 commits (v1 feature, v2 feature, v3 feature).

```
[ec2-user@ip-172-31-15-12 release_versioning]$ ls
README.md
[ec2-user@ip-172-31-15-12 release_versioning]$ echo "Version 1 feature" > app.txt
git add app.txt
git commit -m "v1 feature"
[main b2d5bb2] v1 feature
 1 file changed, 1 insertion(+)
 create mode 100644 app.txt
[ec2-user@ip-172-31-15-12 release_versioning]$ echo "Version 2 feature" >> app.txt
git add app.txt
git commit -m "v2 feature"
[main 859db5f] v2 feature
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-15-12 release_versioning]$ echo "Version 3 feature" >> app.txt
git add app.txt
git commit -m "v3 feature"
[main 5fae278] v3 feature
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-15-12 release_versioning]$
```

git log –oneline

```
[ec2-user@ip-172-31-15-12 release_versioning]$ git log --oneline
5fae278 (HEAD -> main) v3 feature
859db5f v2 feature
b2d5bb2 v1 feature
cba6d0b (origin/main, origin/HEAD) first commit
[ec2-user@ip-172-31-15-12 release_versioning]$
```

Step 3. Create tags and verify

git tag v1.0 b2d5bb2

git tag v2.0 859db5f

git tag v3.0 5fae278

git tag -l

```
[ec2-user@ip-172-31-15-12 release_versioning]$ git tag v1.0 b2d5bb2
[ec2-user@ip-172-31-15-12 release_versioning]$ git tag v2.0 859db5f
[ec2-user@ip-172-31-15-12 release_versioning]$ git tag v3.0 5fae278
[ec2-user@ip-172-31-15-12 release_versioning]$ git tag -l
v1.0
v2.0
v3.0
[ec2-user@ip-172-31-15-12 release_versioning]$
```

Step 4. Push main to GitHub

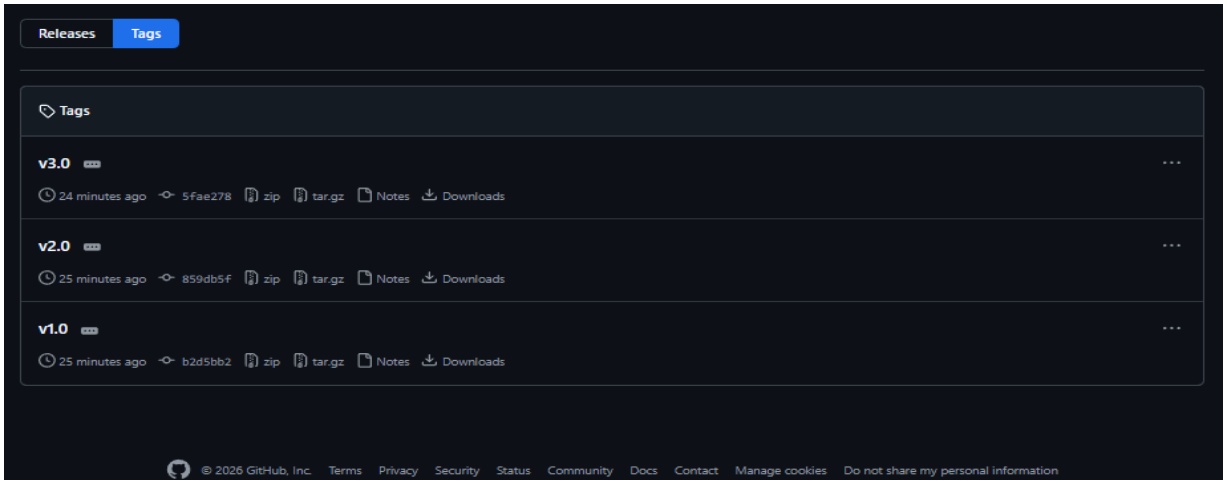
```
[ec2-user@ip-172-31-15-12 release_versioning]$ git push -u origin main
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 2 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), 844 bytes | 844.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/devivaraprasadgannavarapu/release_versioning.git
   cba6d0b..5fae278  main -> main
branch 'main' set up to track 'origin/main'.
[ec2-user@ip-172-31-15-12 release_versioning]$
```

Step 5. Push tags to GitHub

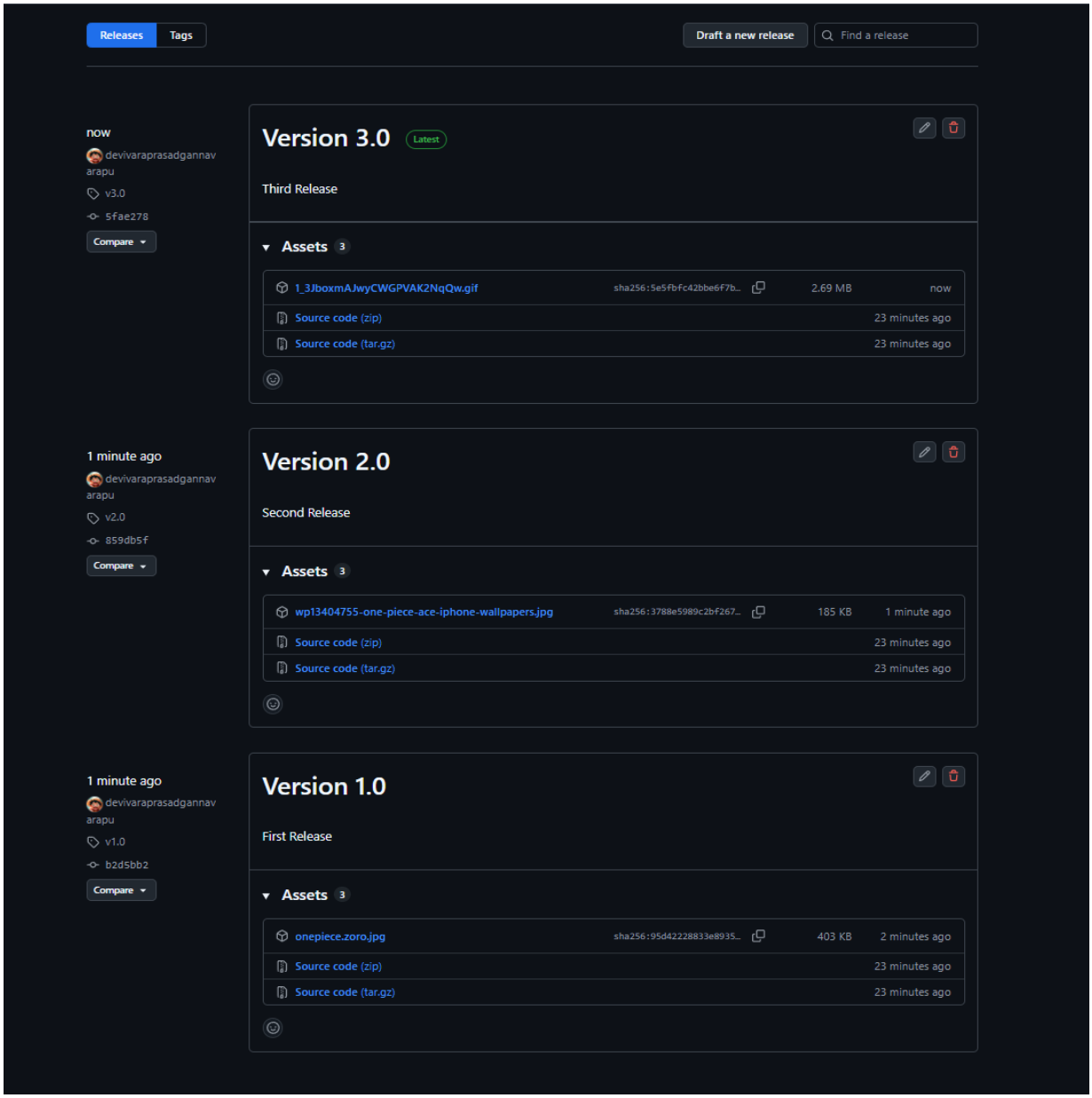
git push origin --tags

```
[ec2-user@ip-172-31-15-12 release_versioning]$ git push origin --tags
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/devivaraprasadgannavarapu/release_versioning.git
 * [new tag]         v1.0 -> v1.0
 * [new tag]         v2.0 -> v2.0
 * [new tag]         v3.0 -> v3.0
[ec2-user@ip-172-31-15-12 release_versioning]$
```

Tags in GitHub



Created Releases in GitHub

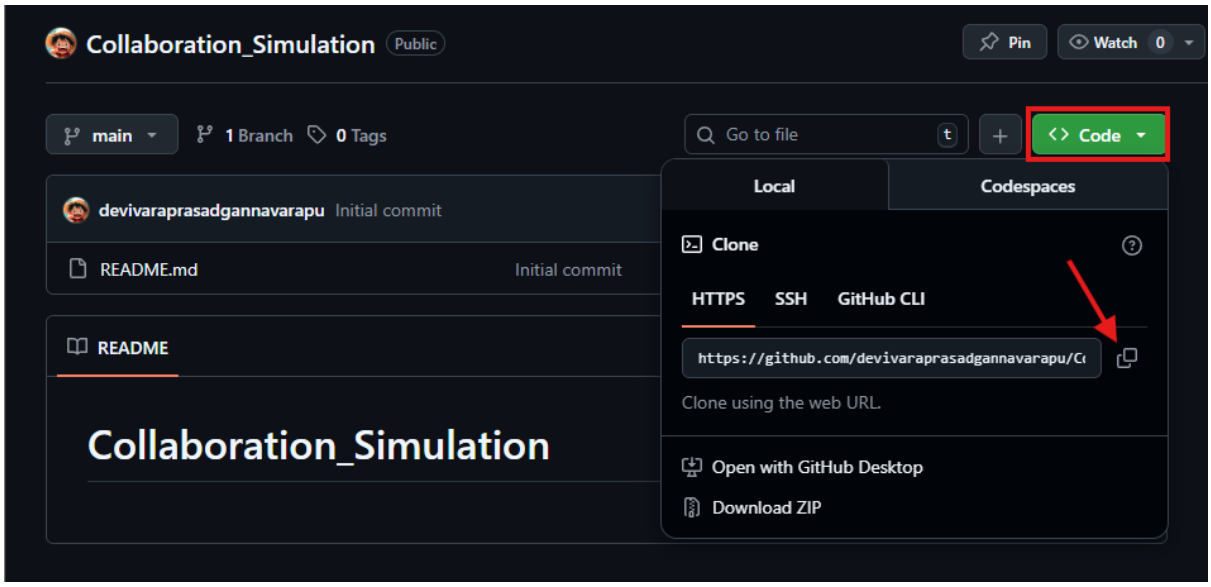


Task7: Git Collaboration Simulation

- Create a new repo on GitHub and clone it locally.
- Create a branch feature/api.
- Make some commits, push to GitHub, and open a Pull Request (PR).
- Merge the PR using GitHub UI.
- Pull changes back to your local repo and verify.
- Share link to the PR and output of git branch -a.

Step 1. Create a new GitHub repository.

Create a repository **Collaboration_Simulation** in GitHub



Step 2. Clone it to local.

git clone https://github.com/devivaraprasadgannavarapu/Collaboration_Simulation.git

```
[ec2-user@ip-172-31-15-12 ~]$ git clone https://github.com/devivaraprasadgannavarapu/Collaboration_Simu
lation.git
Cloning into 'Collaboration_Simulation'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
[ec2-user@ip-172-31-15-12 ~]$
```

Step 3. Create new branch (feature_api) and switch to that branch

git checkout -b feature_api

```
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git checkout -b feature_api
Switched to a new branch 'feature_api'
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$
```

Step 4. Make commits

Add a new file api.sh and made 2 commits

```
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ echo "added changes" >> api.sh
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git add .
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git commit -m "first commit C1"
[feature_api 278447f] first commit C1
 1 file changed, 1 insertion(+)
 create mode 100644 api.sh
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ echo "api.sh modified" >> api.sh
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git add .
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git commit -m "second commit C2"
[feature_api f82f8c8] second commit C2
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$
```

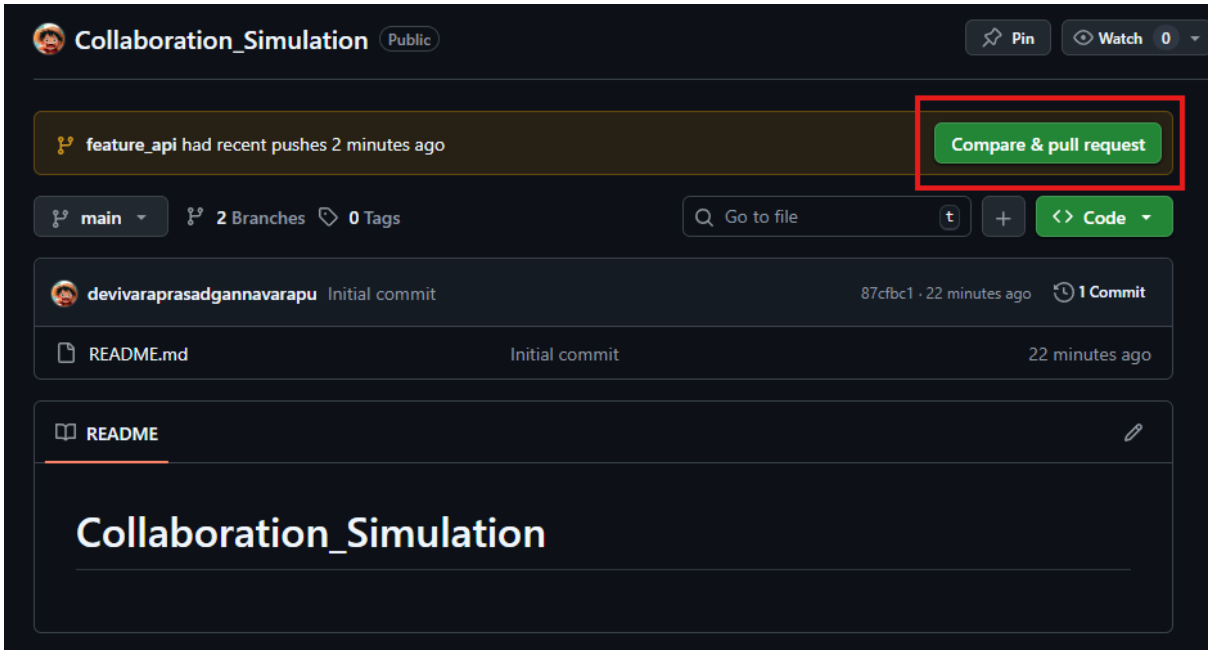
Step 5. Push branch to GitHub

git push origin feature_api

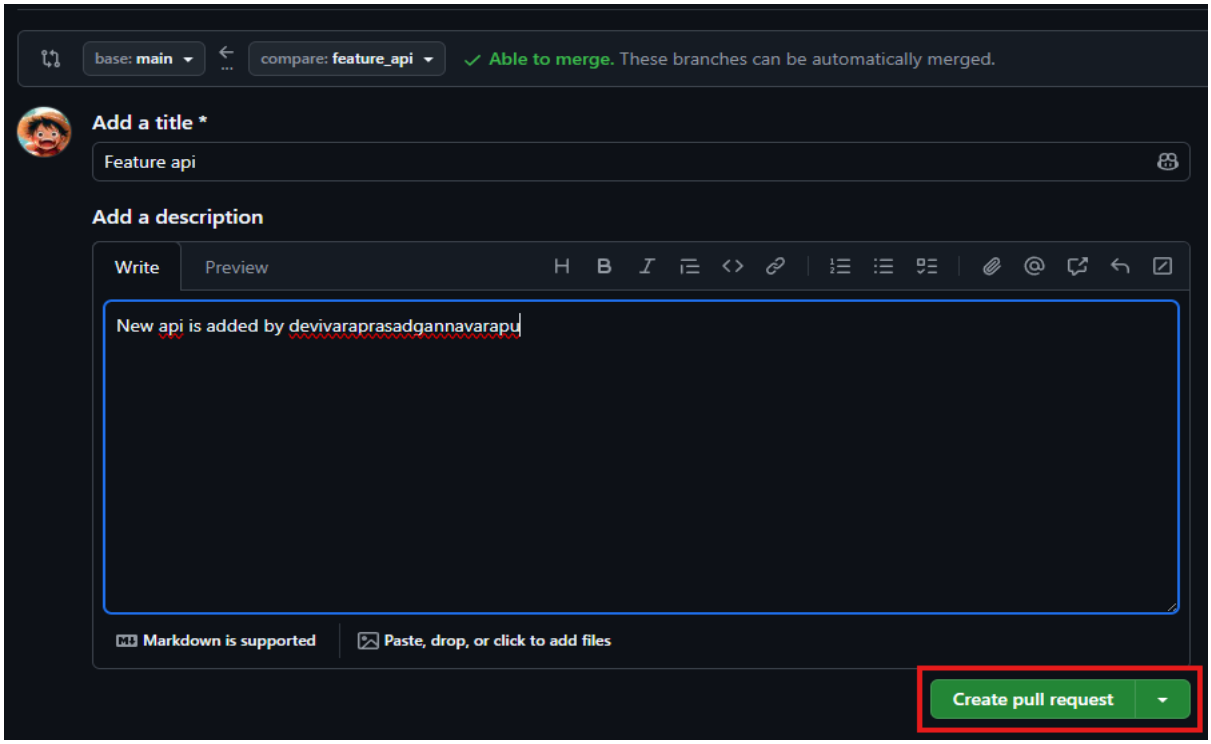
```
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git push origin feature_api
Username for 'https://github.com': devivaraprasadgannavarapu
Password for 'https://devivaraprasadgannavarapu@github.com':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 575 bytes | 575.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature_api' on GitHub by visiting:
remote:      https://github.com/devivaraprasadgannavarapu/Collaboration_Simulation/pull/new/feature_api
remote:
To https://github.com/devivaraprasadgannavarapu/Collaboration_Simulation.git
 * [new branch]      feature_api -> feature_api
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$
```

Step 6. Create and Merge Pull Request (in GitHub)

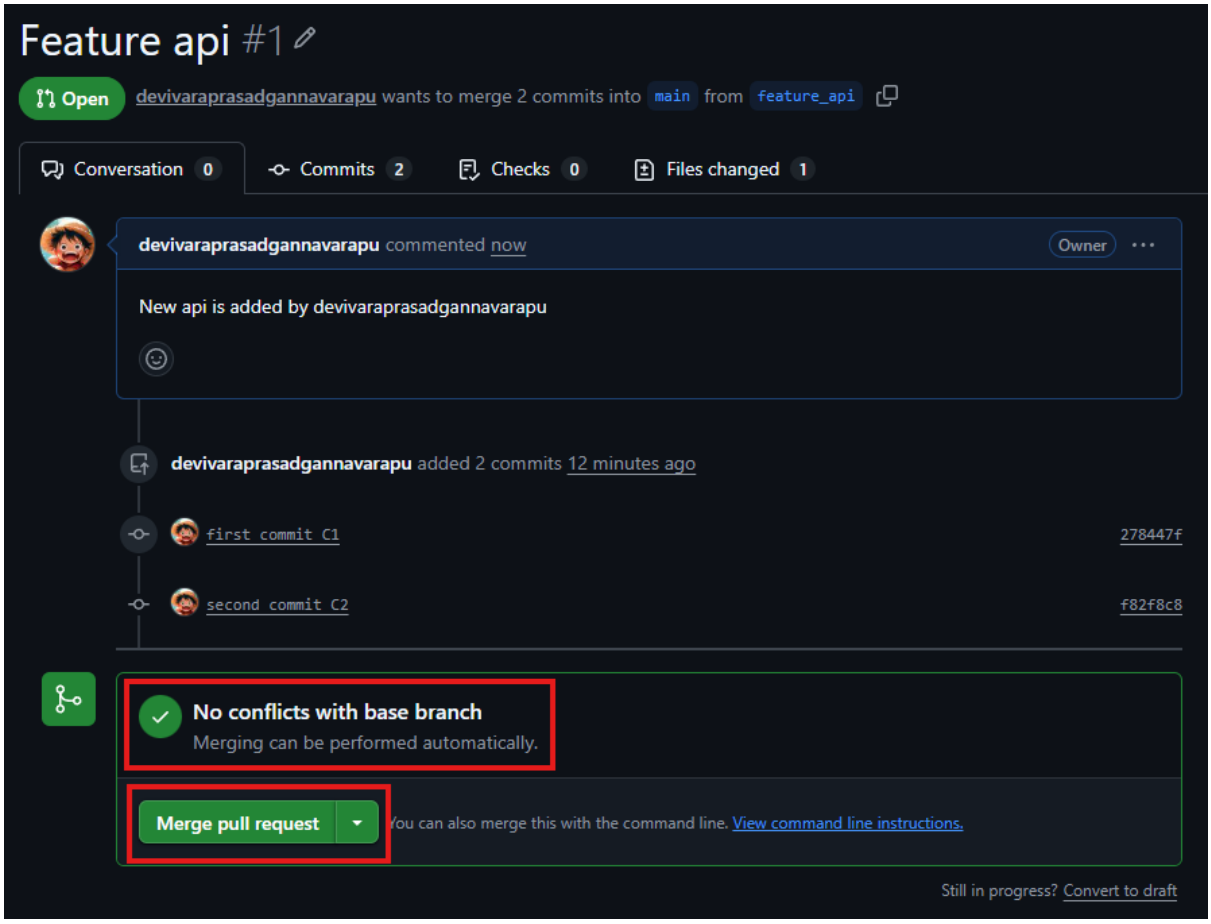
Open compare and pull

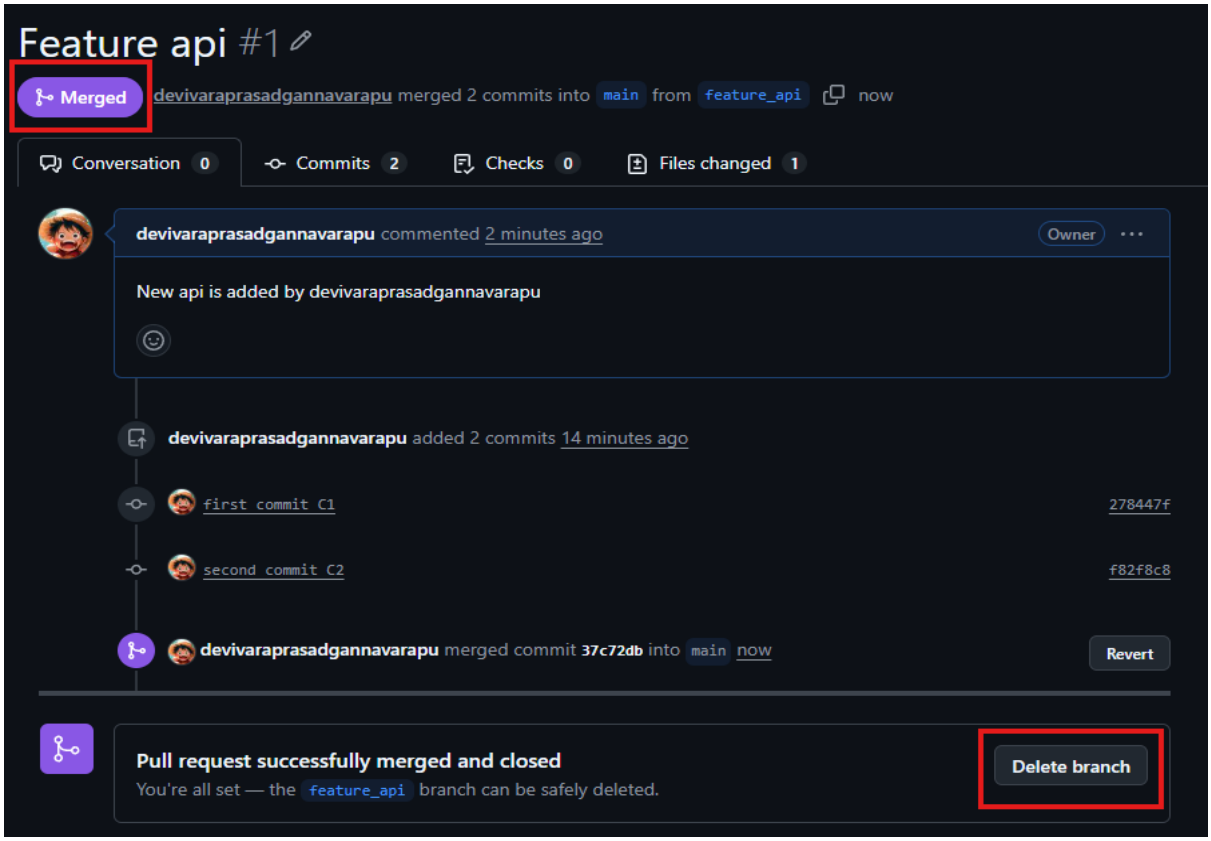


Create pull request



Merge the PR in to the **main branch** using the GitHub





Step 7. Pull changes Locally and verify

After merging, pull updates from GitHub

git switch main

git pull origin main

git branch -a

```
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git pull origin main
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 893 bytes | 893.00 KiB/s, done.
From https://github.com/devivaraprasadgannavarapu/Collaboration_Simulation
 * branch            main       -> FETCH_HEAD
   87cfbc1..37c72db  main       -> origin/main
Updating 87cfbc1..37c72db
Fast-forward
 api.sh | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 api.sh
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$ git branch -a
  feature_api
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature_api
  remotes/origin/main
[ec2-user@ip-172-31-15-12 Collaboration_Simulation]$
```

Assignment 2:

Tasks to Complete:

1. Setup Tomcat

* Download and install Apache Tomcat.

* Start the Tomcat server.

* Verify Tomcat is running by opening http://localhost:8080 in your browser.

2. Build the Application (Maven)

* Open your terminal and navigate to your sample Java web project. (If you don't have one, generate a basic one using Maven).

* Run the Maven command to package the application (e.g., mvn clean package).

* Locate the generated .war file in the target/ directory.

3. Deploy the Application

* Copy the .war file.

* Paste the .war file into Tomcat's webapps folder.

4. Verify the Deployment

* Wait a few seconds for Tomcat to unpack and deploy the application.

* Open your web browser and navigate to your application's URL (e.g., http://localhost:8080/your-app-name).

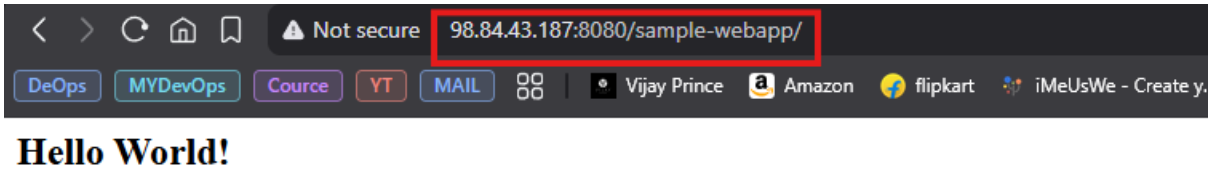
* Confirm the application page loads successfully.

To prove you completed the task, please provide the following screenshots:

Screenshot 1: Your terminal showing the Maven "BUILD SUCCESS" message.

```
[INFO] Processing war project
[INFO] Copying webapp resources [/home/ec2-user/mava/sample-webapp/src/main/webapp]
[INFO] Building war: /home/ec2-user/mava/sample-webapp/target/sample-webapp.war
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.779 s
[INFO] Finished at: 2026-03-13T05:59:01Z
[INFO] ------------------------------------------------------------------------
```

Screenshot 2: Your web browser showing the application running successfully on localhost.



Hello World!

Assignment 3: Deploy a Application in local host

Cloning the Application Repository

First, the project repository is cloned from GitHub.

git clone https://github.com/devivaraprasadgannavarapu/movie-analyzer.git

Navigate to the project directory

cd movie-analyzer

1. Frontend Installation and Setup

The frontend application contains a package.json file which defines the Node.js dependencies required to run the frontend.

To run the frontend application, **Node.js** and **npm** must be **installed**.

```
[ec2-user@ip-172-31-15-12 movie-analyzer]$ cd frontend/
[ec2-user@ip-172-31-15-12 frontend]$ ls
Dockerfile  package-lock.json  package.json  public  server.js  src
[ec2-user@ip-172-31-15-12 frontend]$
```

Install NVM (Node Version Manager)

curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh | bash

```
[ec2-user@ip-172-31-15-12 frontend]$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/mast
er/install.sh | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 16774  100 16774    0     0   815k      0 --:--:-- --:--:-- --:--:--  862k
=> Downloading nvm from git to '/home/ec2-user/.nvm'
```

This command downloads and installs **NVM**, which is used to manage different Node.js versions.

Reload the shell configuration

source ~/.bashrc

This command reloads the shell configuration so the nvm command becomes available.

Install Frontend Dependencies

Navigate to the frontend directory.

cd frontend

Install project dependencies.

npm install package.json

```
[ec2-user@ip-172-31-15-12 frontend]$ npm install package.json
npm warn deprecated package.json@2.0.1: Use pkg.json instead.
npm warn deprecated xss-clean@0.1.4: Package no longer supported. Contact Support at http
s://www.npmjs.com/support for more info.

added 84 packages, changed 286 packages, and audited 1427 packages in 1m

271 packages are looking for funding
  run `npm fund` for details

51 vulnerabilities (15 low, 9 moderate, 21 high, 6 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
[ec2-user@ip-172-31-15-12 frontend]$
```

- npm install reads the package.json file.
- It downloads and installs all required Node.js libraries.

Running the Frontend Application

Start the frontend server using:

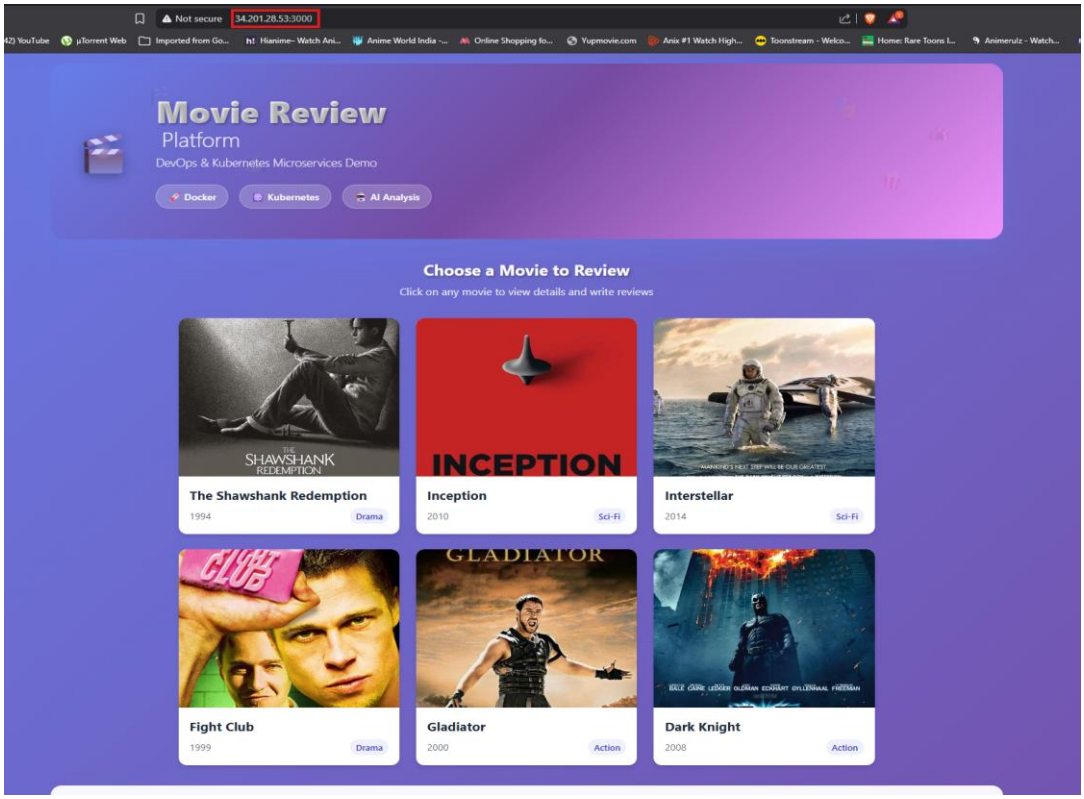
node server.js (or) npm start

```
[ec2-user@ip-172-31-15-12 frontend]$ npm server.js
Unknown command: "server.js"

To see a list of supported npm commands, run:
  npm help
[ec2-user@ip-172-31-15-12 frontend]$ node server.js
🚀 Starting Movie Review Frontend Server
📡 Backend URL: http://localhost:8080
🌐 Frontend Port: 3000
🔧 Environment: NODE_ENV=development
🐳 Running in Docker: Unknown
[HPM] Proxy created: /  -> http://localhost:8080
✅ Frontend server running on port 3000
🎬 Movie Review App is ready!
🔍 Testing backend connectivity to http://localhost:8080...
✅ Backend connectivity test: 404
✅ Backend responded but health data not parsed
(node:2036) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Obje
ct.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
🔄 Proxying: GET /api/admin/health -> http://localhost:8080/api/admin/health
✅ Proxy Response: 404 GET /api/admin/health
🔄 Proxying: GET /api/reviews/shawshank -> http://localhost:8080/api/reviews/shawshank
```

Try :- http:// 34.201.28.53:3000

The frontend server starts on **port 3000** and provides the user interface for the movie analyzer application.



2. Backend Installation and Setup

The backend application is a Java Spring Boot application built using Maven.

Navigate to backend directory

cd ../backend

Build the application using Maven

mvn install

```
[INFO] Installing /home/ec2-user/movie-analyzer/backend/pom.xml to /home/ec2-user/.m2/repositor
y/com/moviereview/movie-review-backend/1.0.0/movie-review-backend-1.0.0.pom
[INFO] Installing /home/ec2-user/movie-analyzer/backend/target/movie-review-backend-1.0.0.jar t
o /home/ec2-user/.m2/repository/com/moviereview/movie-review-backend/1.0.0/movie-review-backend
-1.0.0.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  18.480 s
[INFO] Finished at: 2026-03-13T11:12:24Z
[INFO] ------------------------------------------------------------------------
```

- Maven reads the pom.xml file.
- Downloads required dependencies.
- Compiles the Java code.
- Generates the JAR file inside the target directory.

Generated file:

target/movie-review-backend-1.0.0.jar

Run the Backend Application

java -jar target/movie-review-backend-1.0.0.jar

```
generated-sources  maven-status    movie-review-backend-1.0.0.jar.original
[ec2-user@ip-172-31-15-12 target]$ java -jar movie-review-backend-1.0.0.jar
🚀 Starting Movie Review Backend Service
📋 This service is designed to be resilient to downstream failures
🗄️ Database connection will be tested at runtime, not startup
🤖 Model server connection will be tested per request

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.0)

2026-03-14 01:55:41 [main] INFO  c.moviereview.MovieReviewApplication - Starting MovieReviewApplication v1.0.0
d/target)
2026-03-14 01:55:41 [main] INFO  c.moviereview.MovieReviewApplication - No active profile set, falling back to
2026-03-14 01:55:44 [main] INFO  o.s.d.r.c.RepositoryConfigurationDelegate - Bootstrapping Spring Data JPA rep
2026-03-14 01:55:44 [main] INFO  o.s.d.r.c.RepositoryConfigurationDelegate - Finished Spring Data repository s
2026-03-14 01:55:46 [main] INFO  o.s.b.w.e.tomcat.TomcatWebServer - Tomcat initialized with port 8080 (http)
2026-03-14 01:55:46 [main] INFO  o.a.catalina.core.StandardService - Starting service [Tomcat]
2026-03-14 01:55:46 [main] INFO  o.a.catalina.core.StandardEngine - Starting Servlet engine: [Apache Tomcat/10
2026-03-14 01:55:47 [main] INFO  o.a.c.c.C.[Tomcat].[localhost].[/] - Initializing Spring embedded WebApplicat
2026-03-14 01:55:47 [main] INFO  o.s.b.w.s.c.ServletWebServerApplicationContext - Root WebApplicationContext:
2026-03-14 01:55:47 [main] INFO  o.h.jpa.internal.util.LogHelper - HHH000204: Processing PersistenceUnitInfo [
2026-03-14 01:55:47 [main] INFO  org.hibernate.Version - HHH000412: Hibernate ORM core version 6.6.15.Final
2026-03-14 01:55:47 [main] INFO  o.h.c.i.RegionFactoryInitiator - HHH000026: Second-level cache disabled
```

- java -jar runs the compiled backend application.
- The backend server runs on port 8080.

3. Model Service Setup (Python)

The application also contains a **Python-based sentiment analysis model**.

Navigate to the model directory

cd ../model

Create Python Virtual Environment

python -m venv myenv

```
[ec2-user@ip-172-31-15-12 movie-analyzer]$ python -m venv myenv
[ec2-user@ip-172-31-15-12 movie-analyzer]$ source myenv/bin/activate
((myenv) ) [ec2-user@ip-172-31-15-12 movie-analyzer]$ |
```

This command creates an isolated Python environment for the project.

Activate Virtual Environment

source myenv/bin/activate

```
[ec2-user@ip-172-31-15-12 movie-analyzer]$ python -m venv myenv
[ec2-user@ip-172-31-15-12 movie-analyzer]$ source myenv/bin/activate
((myenv) ) [ec2-user@ip-172-31-15-12 movie-analyzer]$ |
```

Activates the virtual environment so that Python packages are installed only for this project.

Install Python Dependencies

pip install -r requirements.txt

```
((myenv) ) [ec2-user@ip-172-31-15-12 model]$ pip install -r requirements.txt
Collecting flask==3.1.1 (from -r requirements.txt (line 1))
  Downloading flask-3.1.1-py3-none-any.whl.metadata (3.0 kB)
Collecting textblob==0.18.0 (from -r requirements.txt (line 2))
  Downloading textblob-0.18.0-py3-none-any.whl.metadata (4.5 kB)
Collecting nltk==3.9.1 (from -r requirements.txt (line 3))
  Downloading nltk-3.9.1-py3-none-any.whl.metadata (2.9 kB)
```

This installs all Python libraries required for the model service.

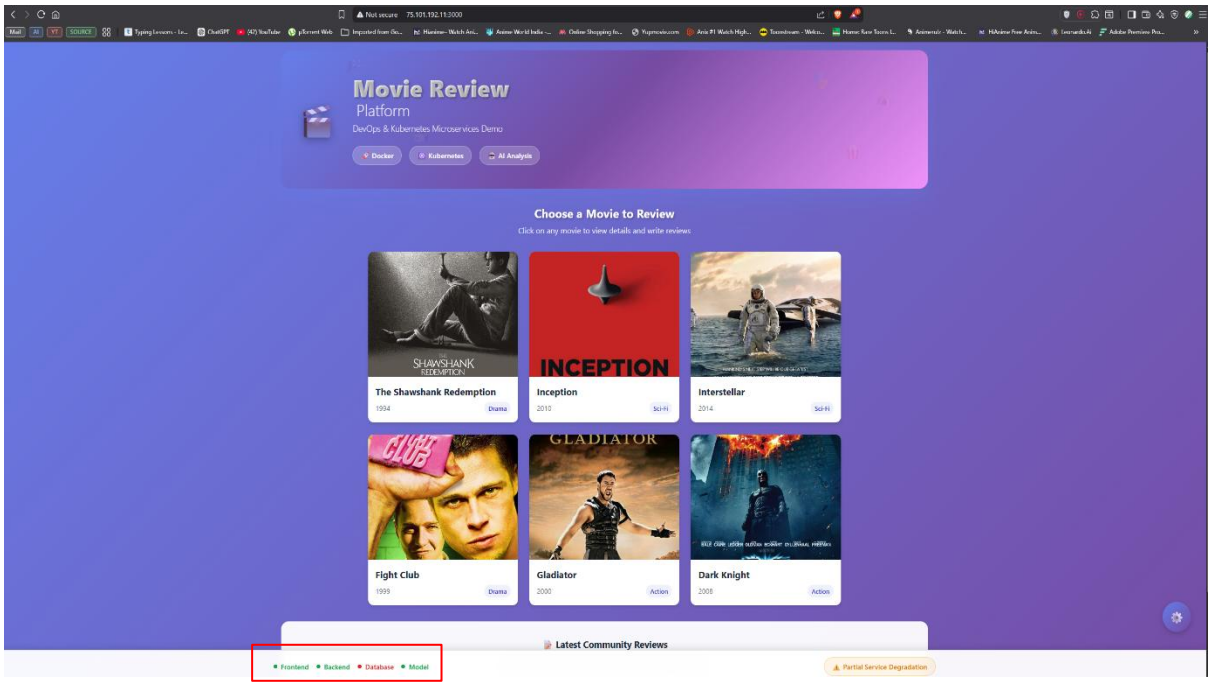
Running the Python Model Service

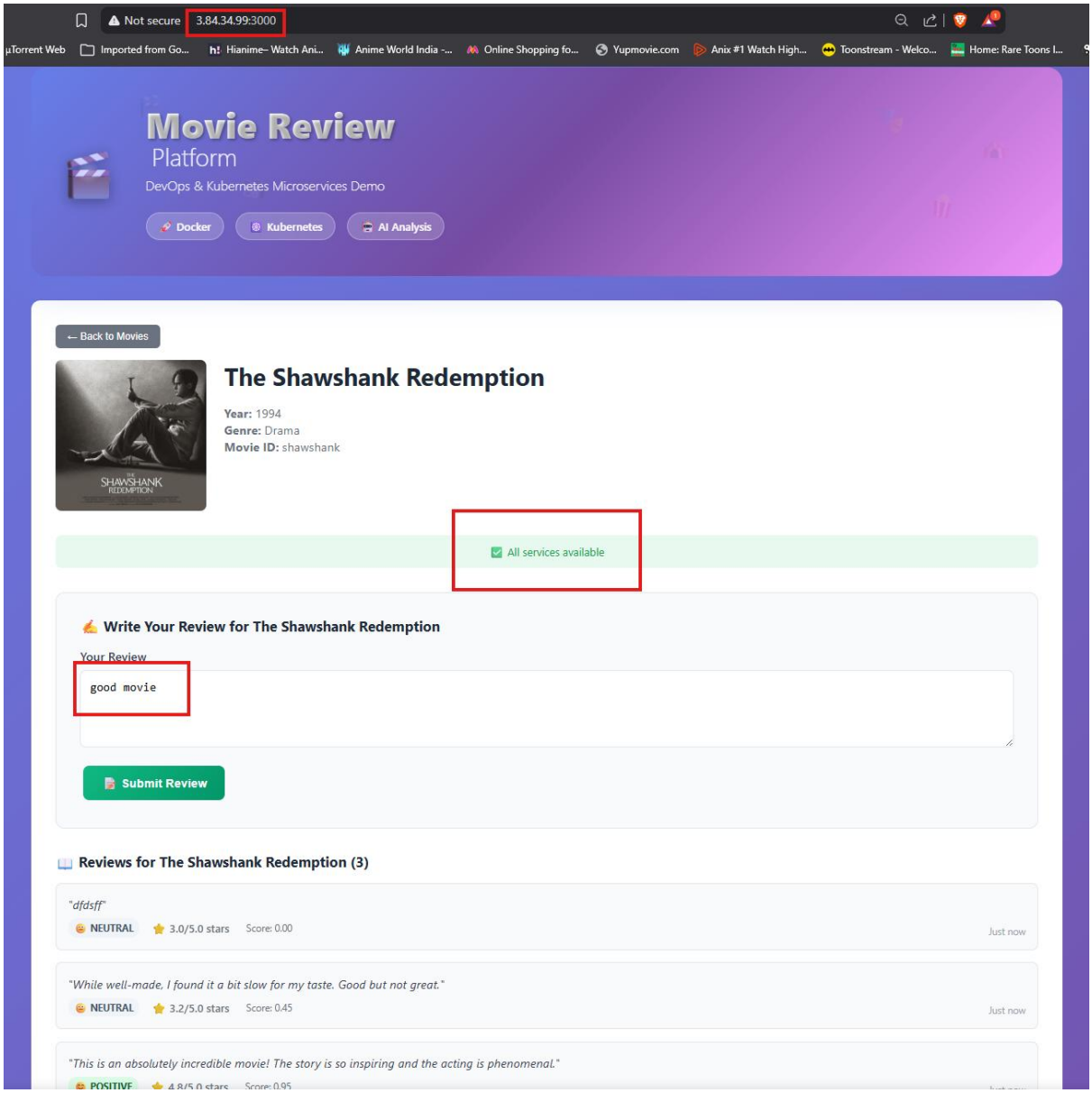
Start the model server:

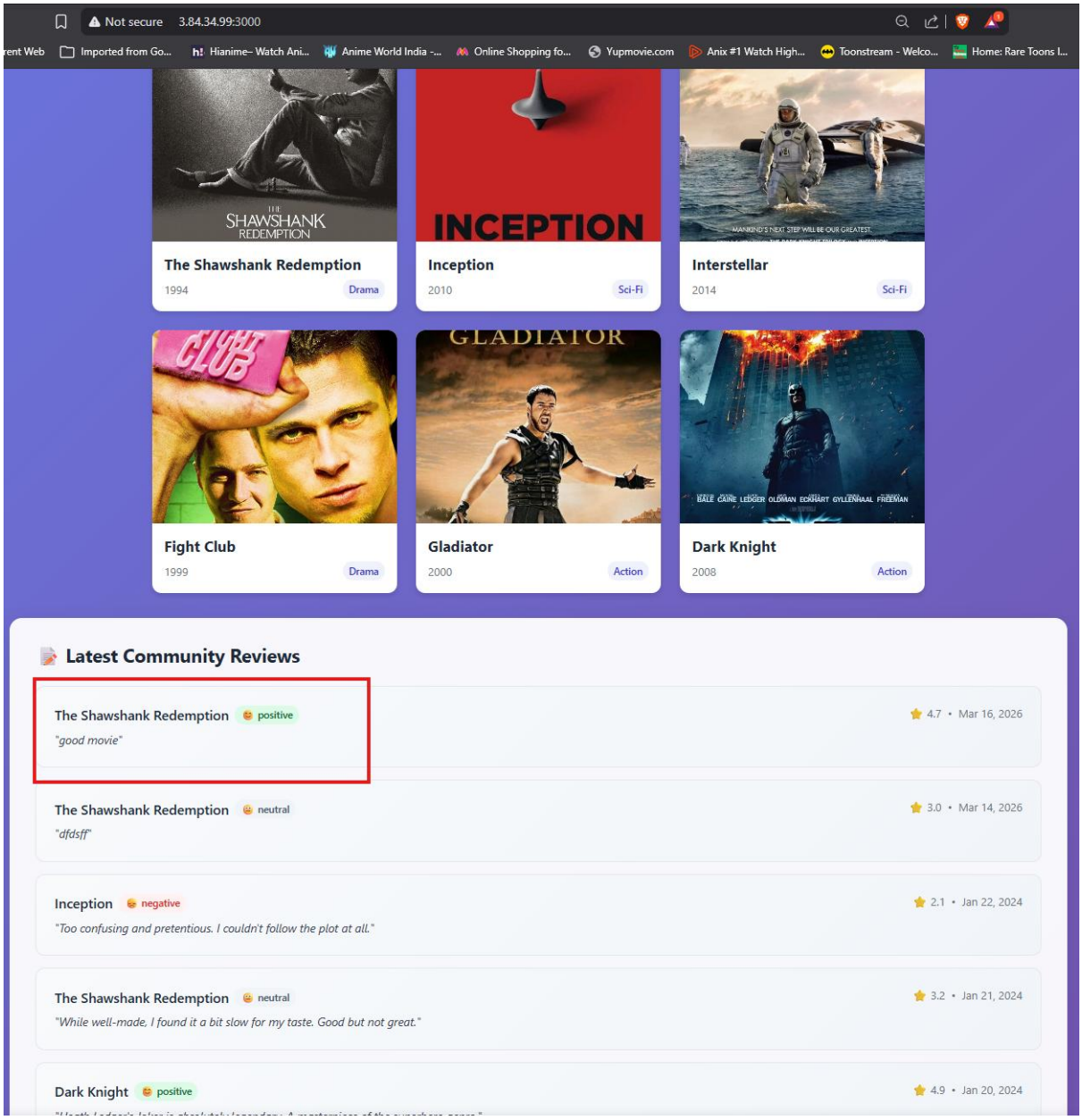
python app.py

```
((myenv) ) [ec2-user@ip-172-31-15-12 model]$ python app.py
🚀 Starting Movie Review Sentiment Analysis Service
🌐 Model Server Port: 5000
📊 Features: Sentiment Analysis + Rating Generation
🔧 Admin: Health toggle endpoint available
✅ NLTK data ready
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.31.15.12:5000
Press CTRL+C to quit
```

- This starts a Flask server.
- The service runs on port 5000.
- It performs sentiment analysis on movie reviews.




Not secure
3.84.34.99:3000
µTorrent Web
Imported from Go...
Hianime– Watch Ani...
Anime World India -...
Online Shopping fo...
Yupmovie.com
Anix #1 Watch High...
Toonstream - Welco...
Home: Rare Toons I...
Movie Review
Platform
DevOps & Kubernetes Microservices Demo
Docker
Kubernetes
AI Analysis
← Back to Movies
The Shawshank Redemption
Year: 1994
Genre: Drama
Movie ID: shawshank
All services available
Write Your Review for The Shawshank Redemption
Your Review
good movie
Submit Review
Reviews for The Shawshank Redemption (3)
"dfdsff"
NEUTRAL
3.0/5.0 stars
Score: 0.00
Just now
"While well-made, I found it a bit slow for my taste. Good but not great."
NEUTRAL
3.2/5.0 stars
Score: 0.45
Just now
"This is an absolutely incredible movie! The story is so inspiring and the acting is phenomenal."
POSITIVE


Not secure 3.84.34.99:3000
Imported from Go...
Hianime– Watch Ani...
Anime World India -...
Online Shopping fo...
Yupmovie.com
Anix #1 Watch High...
Toonstream - Welco...
Home: Rare Toons I...
The Shawshank Redemption
1994
Drama
Inception
2010
Sci-Fi
Interstellar
2014
Sci-Fi
Fight Club
1999
Drama
Gladiator
2000
Action
Dark Knight
2008
Action
Latest Community Reviews
The Shawshank Redemption positive
"good movie"
4.7 • Mar 16, 2026
The Shawshank Redemption neutral
"dfdsff"
3.0 • Mar 14, 2026
Inception negative
"Too confusing and pretentious. I couldn't follow the plot at all."
2.1 • Jan 22, 2024
The Shawshank Redemption neutral
"While well-made, I found it a bit slow for my taste. Good but not great."
3.2 • Jan 21, 2024
Dark Knight positive
4.9 • Jan 20, 2024